✓ **Done:** View

3. Section A

3.1. Question 1

Scenario

Study the scenario and answer the questions that follow:

You have been hired by a local community learning centre to develop a Python-based Learner Progress Tracking System (LPTS). The system will help staff capture learner details, record assessment marks, determine learner performance, and generate simple progress feedback.

The centre offers short training programmes to learners of different ages. Each learner is registered for a course and completes several assessments. Staff members need a system that can store learner details, calculate averages, determine pass or fail status, and display progress summaries.

The application must be developed in Python and should demonstrate the use of fundamental programming concepts studied in the module, including variables, input/output, arithmetic operations, decision structures, loops, functions, lists, object-oriented programming, inheritance, encapsulation, recursion, exception handling, and simple GUI feedback using message boxes.

You are required to develop a Python application for the Learner Progress Tracking System described above.

Requirements

1. Learner Management

- Create a class called Learner with the following attributes:
  - learner_id
  - name
  - age
  - course
  - marks (stored in a list)
- The system must allow staff to:
  - Add new learners
  - View learner details
  - Update learner details
  - Remove learners from the system

2. Inheritance

- Create a base class called Person with common attributes such as:
  - name
  - age
- Create a derived class called Learner that inherits from Person.

3. Encapsulation

- At least one learner attribute, such as the average mark or learner status, must be protected using encapsulation.

- Access to this value must be controlled using methods.

4. Assessment and Marks

- Each learner must have a list of marks.
- The system must allow staff to:
  - Capture marks for a learner
  - Display all marks for a learner
  - Calculate the learner's average mark
- The system must determine whether the learner:
  - Passed
  - Failed
  - Passed with distinction

Suggested rules:

- Below 50: Fail
- 50 to 74: Pass
- 75 and above: Pass with distinction

5. Decision Structures

- Use if, if-else, and if-elif-else statements to:
  - Validate learner age
  - Check whether marks are valid
  - Determine learner performance category
  - Check whether a learner qualifies for a certificate

6. Repetition Structures

- Use loops in the program, including:
  - A condition-controlled loop for the main menu
  - A for loop to process learner marks
- Include the use of:
  - break
  - continue

7. Functions

Create functions to perform tasks such as:

- Adding a learner
- Entering marks
- Calculating average
- Displaying learner summaries
- Searching for a learner by ID

The program must also include at least one of these functions:

- A predicate function that returns True or False
- A function with a default argument
- A function call that uses a keyword argument

8. Recursion

- Include one recursive function.
- For example, create a recursive function that:
  - Calculates the sum of all marks in a learner's mark list
  - Or counts the number of learners in the system

9. Exception Handling

- Use try-except blocks to handle:
  - Invalid numeric input
  - Invalid mark entries
  - Division by zero where applicable
  - Menu selection errors

10. Lists

- Use a list to store learners.
- Use a list to store marks for each learner.
- The system must be able to:
  - Traverse the lists
  - Search through the lists
  - Update list values where necessary

11. GUI Requirement

- The system may be console-based, but it must include simple GUI feedback using message boxes.
- Use tkinter.messagebox to display messages such as:
  - Successful learner registration
  - Invalid input
  - Pass/fail result
  - Exit confirmation

Note: Full event-driven GUI design is not required. Only simple message boxes should be used where appropriate.

12. Output

The system must display:

- Learner details
- Marks entered
- Average mark
- Performance result
- Whether the learner qualifies for a certificate

13. Documentation

Your program must include:

- Comments at the top of the file indicating:
  - Programmer name
  - Date
  - Program purpose
- Comments for important sections of the code
- Meaningful variable names
- Proper indentation

Deliverable

Develop and submit a working **Python Learner Progress Tracking System** prototype for internal review, together with well-documented source code.

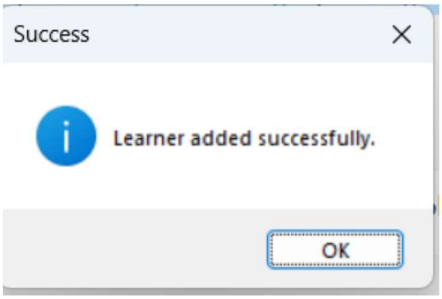
Example of working program

Adding a learner

```
In [1]: %runfile C:/Users/cash/.spyder-py3/temp.py --wdir

===== Learner Progress Tracking System =====
1. Add learner
2. Enter marks
3. View all learners
4. Search learner
5. Update learner
6. Remove learner
7. Show learner result
8. Exit
Enter your choice: 1
Enter learner ID: 1
Enter learner name: lucy manyange
Enter learner age: 18
Enter course name: intro to python
```

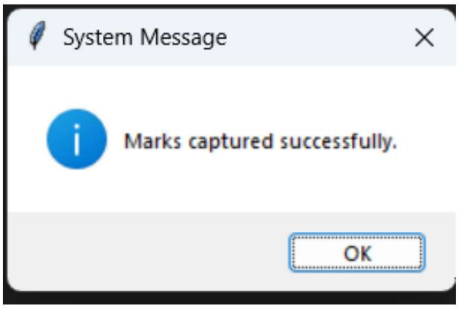
Messagebox showing learner successfully added



Adding Marks

```
===== Learner Progress Tracking System =====
1. Add learner
2. Enter marks
3. View all learners
4. Search learner
5. Update learner
6. Remove learner
7. Show learner result
8. Exit
Enter your choice: 2
Enter learner ID: 1
How many marks do you want to enter? 3
Enter mark 1: 50
Enter mark 2: 76
Enter mark 3: 65
```

Messagebox showing marks added successfully



Viewing Students

```
Enter your choice: 3

----- Learner Summary -----
Learner ID: 1
Name: lucy manyange
Age: 18
Course: intro to python
Marks: [50.0, 76.0, 65.0]
Average: 63.67
Result: Pass
Certificate: Yes
```

Searching for a learner

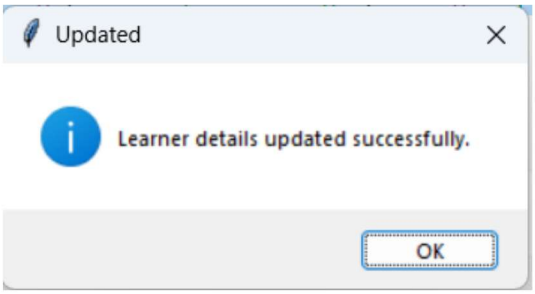
```
Enter your choice: 4
Enter learner ID to search: 1

----- Learner Summary -----
Learner ID: 1
Name: lucy manyange
Age: 18
Course: intro to python
Marks: [50.0, 76.0, 65.0]
Average: 63.67
Result: Pass
Certificate: Yes
```

Updating a learner

```
===== Learner Progress Tracking System =====
1. Add learner
2. Enter marks
3. View all learners
4. Search learner
5. Update learner
6. Remove learner
7. Show learner result
8. Exit
Enter your choice: 5
Enter learner ID to update: 1
Enter new name: Gaby
Enter new age: 19
Enter new course: IT
```

Learner successfully updated

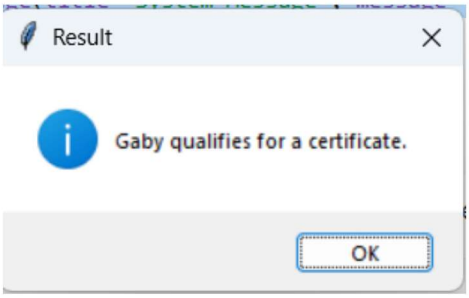


Showing learner results

```
7. Show learner result
8. Exit
Enter your choice: 7
Enter learner ID: 1

Result for Gaby
Average: 63.67
Performance: Pass
```

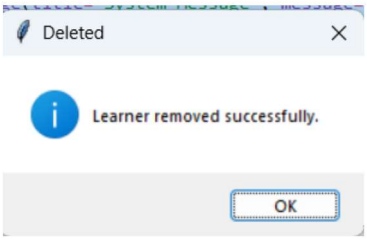
Showing whether student qualifies for certificate



learner

```
===== Learner Progress Tracking System =====
1. Add learner
2. Enter marks
3. View all learners
4. Search learner
5. Update learner
6. Remove learner
7. Show learner result
8. Exit
Enter your choice: 6
Enter learner ID to remove: 1
```

Learner deleted successfully



Exit